

Guardrails

Building agents by building their harness

Lesson 6 of 10 · Production

Where we left off

Lesson 5 lets us **see** everything the agent does.

We can read every command it ran.

After it ran.

The problem with after

The trace is a record.

It does not stop anything.

The command has already run.

Remember the warning

From Lesson 2 on, the agent runs model-written shell commands.

With no confirmation.

Two things can go wrong

- one **action** is harmful
- the **run as a whole** goes on too long

One action

```
rm -rf something it shouldn't.
```

Overwrite a file.

One bad command is enough.

The whole run

The loop never stops asking for tools.

Lesson 3: "A loop with no clear way to end is a bill with no clear way to end."

Guardrails

Checks that can stop the agent before it acts, or before it goes on.

Where does it live?

Guardrails are **built on control flow**.

Why control flow

Control flow decides **whether to go again**.

Control flow sits between a request for a tool and the tool running.

That is where you put a gate.

Two kinds of guard

- a **gate** before each tool
- **limits** at the top of the loop

Plus one more, in the richer file: an **interrupt**.

The worked example

`production/06-guardrails/quark.py`

Lesson 5's `quark.py` plus guards.

83 lines.

Start with the gate

Before a tool runs, the harness gives a verdict.

Three verdicts

- **allow**: run it
- **ask**: the person decides
- **deny**: don't run it

Where the gate sits

In the tool branch.

Before the trace.

Before `subprocess.run`.

So denied commands are not traced

Lesson 5 records what ran.

A denied command never ran.

It never reaches `trace()`.

Four new pieces

- `import re`
- `SAFE`
- `DENY`
- `guard(cmd)`

SAFE

A set of read-only programs: `ls` , `cat` , `wc` , `grep` ...

Runs unasked only if **every** piece between pipes starts with one,

and nothing chains, redirects or expands: no `;` `&` `<` `>` `$` , backtick, newline or `(` .

DENY

A regular expression.

It matches patterns that should never run.

If it matches: **deny**.

guard(cmd)

Takes the command string.

Returns the verdict.

Three outcomes, one function.

The order of checks

1. matches **DENY** ? **deny**
2. only **SAFE** programs, nothing chained or redirected? **allow**
3. otherwise: **ask**

Why default to ask

An unknown command is neither known-safe nor known-bad.

So a person decides.

Safe by default, not by hope.

Asking the person

The harness prints the command.

It waits for an answer.

That's control flow deciding whether the next step runs. The answer is a decision, not a task.

What happens on yes

The command runs.

Everything continues as in Lesson 5.

What happens on no

The command does not run.

But the model asked and is **waiting**.

The tool request still needs an answer.

The refusal goes back as a result

A `tool_result` with `is_error` set.

It says why the command was not run.

Why is_error

The model reads the result.

It learns that this failed and why.

It can try something else.

Refusal is information

Not a crash.

Not silence.

An ordinary tool result, marked as an error, in the slot the result would have filled.

The person can explain

Type `no` , then a reason.

"no, don't touch that folder"

The reason goes back as an instruction.

A guard talking to the model

The person steers it.

Without a new task. Without restarting.

Now the limits

The gate guards each action.

Limits guard the **run**.

Two counters

- `steps` : how many model calls
- `spent` : how many tokens

Both start at zero.

Two constants

- MAX_STEPS
- MAX_TOKENS

The ceilings.

The stop check

At the **top** of the loop.

Before the next model call.

Over a ceiling? Stop.

Why the top

Stopping before the call costs nothing.

Stopping after is a call you already paid for.

Counters reset per task

In chat mode, a new task resets both.

Each task gets its own budget.

Lesson 3 had MAX_STEPS too

`control_flow.py` stopped after 10 steps.

That was a counting loop in one file.

Now it is a guard in the working agent.

Who stops the run

The harness checks the limits.

The harness stops the run.

The model doesn't decide to stop.

Run it

```
uv run production/06-guardrails/quark.py \  
    "how many lines are in README.md?"
```

A command that needs asking

The harness prints it and waits.

You answer `y`.

A command that gets denied

Answer `no, <reason>`.

Watch the model read the reason and change course.

A note on the outputs

The recorded outputs were made by **piping answers into stdin**.

Answers are not echoed.

At the keyboard you will see your own typing.

Seeing a limit

Make a copy with `MAX_STEPS = 2`.

Give it a task that needs more.

It stops after two.

Going further

`quark.py` is one way to do it.

`guardrails.py` is a richer one.

guardrails.py

Built on Lesson 3's `control_flow.py` .

Adds structure to the verdicts.

ALLOWED and DENIED

ALLOWED : a set of programs that may run unmasked.

DENIED : patterns that are always refused, each with its reason.

One command is many programs

```
ls && rm -rf x ; cat y | grep z
```

Checking the first word is not enough.

programs()

Splits a command on:

`&&` `||` `;` `|` and newlines.

Every program in the line gets checked.

verdict()

A `DENIED` pattern anywhere: **deny**.

A redirect or `$`: **ask**.

Any program not in `ALLOWED`: **ask**.

Only a command made entirely of allowed programs runs unasked.

ask() with more answers

- `y` : allow once
- `always` : allow this from now on
- anything else: a **reason**, sent to the model

Dollar limit

`MAX_DOLLARS` .

Uses a `PRICE` dict of **example rates**.

They show which run cost more, not your bill.

Repeat limit

```
MAX_REPEATS .
```

The same command again and again?

Stop. The model is stuck.

Seeing the dollar limit

Make a copy with `MAX_DOLLARS = 0.004`.

It stopped at \$0.0041 against \$0.004.

The limit is checked before each call, so a run can end a little over.

Interrupt

Press **Ctrl-C** while it works.

Only in `guardrails.py` .

The tricky part of interrupting

The model may have asked for tools.

Every `tool_use` needs a `tool_result`.

Stop in the middle and the next request is **malformed**.

The handler

On `KeyboardInterrupt`, it answers each unanswered tool request.

Then it asks you: **what now?**

After the interrupt

You type a new instruction.

The model carries on, with that.

Not a crash. A hand-back.

Why interrupt is control flow

It decides whether the loop goes on.

And who gets the next turn.

The person, this time.

Policy languages

Open Policy Agent and NeMo Guardrails.

They express rules like these in their own language.

This layer, packaged.

What guardrails can be

- **what** is checked: command, path, spend
- **who** decides: code, person, another model
- **when**: before an action, or between steps
- **what happens**: stop, ask, tell the model

What to take away

Rule: guardrails are checks in control flow that decide whether an action runs and whether the loop goes on.

Notice what guardrails never do

They never change how a request is built or sent.

A refusal goes back as an ordinary tool result.

They decide **whether** the next step happens.

What's missing

A guard decides whether a command runs.

It does not limit **what an allowed command can reach**.

An allowed command still has the whole machine.

Next: Sandboxing

Lesson 7 limits what a command can reach.